How complex is your neural network? Counting parameters is the wrong answer

Blog post draft, English. Benchmark results and the release link are placeholders until they exist.

Classical statistics gives us standard tools for choosing a model: AIC, BIC, R^2 and their variations. They work well for the models they were built for. Hand them a large neural network and most of them stop making sense.

The parameter-count trap

Ask how big a neural network is and you will hear a number of parameters. Ask why a network with millions of parameters works on data it has never seen, and that number stops helping. Modern networks routinely have more parameters than training examples and still generalize. Parameter count tells you the size of a model, not how complex it really is.

Think of two books. One has a thousand pages, but every page repeats the same sentence. The other is a single page of random characters. The thick book is simple: a short note explains all of it. The thin one is complex: nothing short describes it. Length and complexity are different things, for books and for networks.

Complexity as description length

That intuition has a precise form. Ask instead: **how many bits does it take to describe this model together with the data it learned from?** If a short description is enough, the model found real structure in the data. If you need a long one, it is mostly memorizing. This is the minimum description length (MDL) principle.

Statisticians reach a close cousin of the same idea from another side. In Bayesian terms, a model that fits the data too flexibly pays a penalty in its evidence, and that penalty plays the role of complexity. The two views meet in the code length of a Bayesian mixture, so techniques from both camps can speak about the same thing.

Many roads, different meanings

Over the years people have turned this idea into many concrete measures, and they do not all answer the same question:

- **Information criteria** such as AIC and BIC balance how well the model fits against a penalty for its flexibility. They are simple and cheap, and they lean on assumptions that large networks often break.
- **Code lengths** go back to the bits. They ask what it costs to transmit the model and the data, either by describing the weights directly or by encoding the data as a stream, one block at a time, with a model that keeps learning.
- **Risk bounds** ask a different question again: if the trained network can be compressed hard without changing what it does, how much can we guarantee about its error on new data?

These measures speak related languages, but they are not the same language. One reports a score on its own scale, another a number of bits, another a guarantee that holds only under conditions. A single number that ranks them all would be a trap. We keep one rule in mind: **one calling convention does not imply one scale or a universal ranking.**

What we are building

Our project, **DL Complexity**, is a PyTorch library that lets you ask each of these measures in the same way, while keeping what each one means. Every answer comes with its unit, the assumptions behind it and what was needed to compute it. Nothing is trained behind your back, and the library never tells you that a score from one family beats a score from another.

What we want to find out

A good measure of complexity should say something about generalization. So we will train many small networks that differ in width, depth, regularization and the amount of data they see, first on synthetic data and then on MNIST and Fashion-MNIST. For each one we compute every measure and ask a simple question: when the measure says one model is more complex than another, does that model generalize worse? We compare how measures order models, not their raw values. The humble parameter count is the baseline to beat.

[Headline figure: a dot plot with one row per measure. The x axis is the rank correlation between the measure and the generalization gap on Fashion-MNIST, with confidence intervals across seeds; a dashed line marks the parameter-count baseline; rows are sorted by correlation and coloured by family: information criteria, code lengths, risk bounds.]

[two or three sentences on the main result: which measure tracked generalization best, where the parameter count was beaten, and how the picture changed with more training data.]

[one sentence on cost: which measures are cheap enough to run on every model you train, and which are for special occasions.]

Why it matters

If this works, a complexity measure becomes a practical instrument rather than a theory exercise. It could tell you when a model is memorizing rather than learning, help you compare architectures beyond their size, and give you an idea of how much data a model really needs. If it does not work for some measures, that is useful to know too, and we will say so.

Follow along

The library will be released as the open-source Python package `dl_complexity`. **[TBD: install command and public repository link once the package is released.]** The project lives at github.com/intsystems/dl-complexity.